

STRINGS 2

**"We achieve more
when we chase the
dream instead of
the competition."**

Simon Sinek

Good
Evening



Today's content

- Longest substring with all distinct character
- String permutation check
- No. of substrings that are permutation of another string
- Group Anagrams

01. Length of longest substring with all distinct character

$S_1 = a b c a b c d d$ Ans = 4

$S_2 = s i p p i e r$ Ans = 4

$S_3 = "a a a a"$ Ans = 1

Idea 1 → Generate all substrings & check if its distinct or not, if distinct → get max length.

```
int longest (String str)
```

```
    ans = 0;
```

```
    for (i = 0; i < n; i++)
```

TC: $O(n^3)$

```
        for (j = i; j < n; j++)
```

SC: $O(n)$

```
            // substring [i...j]
```

```
            HashSet < Characters > hs;
```

```
            for (k = i; k ≤ j; k++) {
```

```
                | hs.add(str.charAt(k));
```

```
            }
```

```
            if (hs.size() == j - i + 1) {
```

length of substring

```
                | ans = Math.max(ans, j - i + 1);
```

```
            }
```

```
    return ans;
```

```
}
```

Idea 2 → For every index, check the length of longest substring that we can generate from that particular index

Str = { 0 1 2 3 4 5 6 7
a b a c e a f f }

↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
len=2 ↓ len=3 4 3 ↓ ↓ ↓
len=4 2 1 1

substring = { b a c e }

```
int longest (String str) {
```

```
    for (i=0; i<n; i++)
```

TC: $O(n^2)$

SC: $O(n)$

```
        HashSet <Character> hs;
```

```
        for (j=i; j<n; j++) {
```

```
            char ch = str.charAt(j);
```

```
            if (hs.contains(ch) == false) {
```

```
                | hs.add(ch);
```

```
                |
```

```
            } else {
```

```
                | break;
```

```
        }
```

ans = Math.max(ans, hs.size());

2

3

Idea 3

0 1 2 3 4 5 6 7 8 9 10 11 12 13
str = a b c g h e g k l m h a b k
 ↑ ↑
 P₁ P₂

Valid

(0, 0)

(0, 1) (1, 1)

(0, 2) (1, 2)

(0, 3) (1, 3)

(0, 4) (1, 4)

(0, 5) (1, 5)

HashSet {~~x~~, ~~x~~, ~~x~~, ~~x~~, h, e }
 g, k, l, m

(0, 6) → (1, 6) → (2, 6) → (3, 6) → (4, 6) → (4, 7)
 str[0]* str[1]* str[2] str[3] str[6]

(4, 7) → (4, 8) → (4, 9) → (4, 10)
 ↓
 start dropping
 characters from
 left side

```
int longestdis (String str)
```

```
HashSet <Character> hs ;
```

```
int P1 = 0 , P2 = 0
```

```
while ( P2 < n )
```

```
    if ( hs.contains ( str.charAt ( P2 ) ) == false )
```

```
        hs.add ( str.charAt ( P2 ) );
```

```
        P2 = P2 + 1 ;
```

```
        ans = Math.max ( ans, hs.size () );
```

```
    }
```

```
    else {
```

```
        hs.remove ( str.charAt ( P1 ) );
```

```
        P1 = P1 + 1 ;
```

```
    }
```

```
}
```

```
return ans;
```

```
}
```

TC: $O(n)$

SC: $O(n)$

10:13 → 10:22 pm

Q Given 2 strings S_1 & S_2 of equal length
check if they are permutations of each other
Permutation: Freq of all chars must be same in
both strings

Eg:-

S_1		S_2
cat	True	act
abacb	True	aabcb
abbca	False	accab
atca	False	cat

Idea 1 $\rightarrow$ Sort & compare both strings

S_1 abbca	$\longrightarrow$	aabbc	Tc: $O(n \log n + n)$ Sc: $O(n)$
$S_2 =$ accab	$\longrightarrow$	aabcc	

check if we have
to convert string
to array to sort it

Idea 2 = Two hashMap

- $\rightarrow$ Insert all the char of S_1 in HM_1
- $\rightarrow$ Insert all the char of S_2 in HM_2
- $\rightarrow$ Compare frequencies of characters from
 HM_1 & HM_2

Idea 3 • Using One hashMap

- Insert all the char of S_1 in HM
- Iterate on S_2 & reduce the frequency of characters
- Iterate on HM & check if all the characters has frequency = 0

$S_1 = \text{"abc"}$
 $S_2 = \text{"cab"}$ } n length

Pseudocode →

HashMap <Character> hm = new HashMap <>();

// inserted S_1 to hm

for (i=0; i<n; i++){

 char ch = S_1 .charAt(i);

 if (hm.containsKey(ch) == false){

 { hm.put(ch, 1);
 }

 else {

 int of = hm.get(ch);

 hm.put(ch, of + 1);

 }

// reduce freq according to S_2

```
for (i=0; i<n; i++)
```

```
    char ch = S2.charAt(i);
```

```
    if (hm.containsKey(ch) == false) {
```

```
        return false
```

```
    }
```

```
    else {
```

```
        int of = hm.get(ch);
```

```
        hm.put(ch, of - 1);
```

```
    }
```

TC: $O(n)$

SC: $O(n)$

```
for (char key : hm.keySet()) {
```

```
    if (hm.get(key) != 0)
```

```
        return false;
```

```
    }
```

```
    }
```

```
return true;
```

Idea 4 = Using two hashmap

$S_1 = \{a, a, a\}$

HM1 = {a: 3}

$S_2 = \{a, a, b\}$

HM2 = {a: 2, b: 1}

count of matching char = 1 + 1

2 ≠ 3
false

$S_1 = \{a \ c \ b \ a \ c \ b \ a\}$

HM1 = { a:3 b:2 c:2 }

$S_2 = \{a \ b \ c \ c \ b \ c \ b\}$

HM2: { a:1 b:3 c:3 }

count no. of matching char : 1 + 1 + 1 + 1 + 1

5 ≠ 7 false

Note 1 → Extra char won't going to increase count

Note 2 → If 2 strings are permutation of each other
then count == N

boolean permutation (String s_1 , String s_2)

HashMap <Character, Integer> hm_1

// inserted s_1 to hm

for ($i=0$; $i<n$; $i++$) {

char $ch = s_1.charAt(i)$;

if ($hm_1.containsKey(ch) == false$) {

$hm_1.put(ch, 1)$;

}

else {

 int $of = hm_1.get(ch)$;

$hm_1.put(ch, of + 1)$;

}

}

TC: $O(n)$

SC: $O(n)$

HashMap <Character, Integer> hm_2

int match = 0

for ($i=0$; $i<N$; $i++$)

char $ch = s_2.charAt(i)$

if ($hm_2.containsKey(ch) == false$) {

$hm_2.put(ch, 1)$;

}

else {

 int $of = hm_2.get(ch)$;

$hm_2.put(ch, of + 1)$;

}

```

    if (hm2.get(ch) ≤ hm1.get(ch))
        |
        | match = match + 1;
        |
        3
    2

```

if (match == N) return true
 else return false;

03. Count no. of substrings of A, that are permutations of B.

A_N:

0	1	2	3	4	5	6	7
a	b	c	b	a	b	a	c

Ans = 3

B_K: a b c

Idea 1 → Generate all the substrings of A of size k
 & then we will check if it is permutation
 of B or not.

TC : # No. of substring of len = k * TC to check if substring
 is permu of B

$$TC = \underline{O(n - k + 1)} * K = O(n^2)$$

Idea 2 $\rightarrow$ fixed subarr length $\rightarrow$ sliding window

$B_k = \{a\ b\ a\ c\ b\}$ $\xrightarrow{\hspace{1cm}}$ HM: $\{a:2\ b:2\ c:1\}$

$A_n = \{a\ c\ a\ c\ a\ b\ b\ c\ a\}$

0 1 2 3 4 5 6 7 8

substring		HM2	match	ans
[0 4]	remove	add $\{a:3\ c:2\}$	$3 \neq 5$	+0
[1 5]	$A[0]$	$A[5]$ $\{a:2\ b:1\ c:2\}$ valid char	$3+1 \neq 5$	+0
[2 6]	$A[1]$	$A[6]$ $\{c:1\ b:2\ a:2\}$	$4+1=5$	+1
[3 7]	$A[2]$	$A[7]$ $\{a:1\ c:2\ b:2\}$	$5-1 \neq 5$	+0
[4 8]	$A[3]$	$A[8]$ $\{c:1\ a:2\ b:2\}$	$4+1=5$	+1
				<u>Ans = +2</u>

HashMap <Character, Integer> hm₁

// inserted s₁ to hm

```
for (i=0; i<K; i++) {  
    char ch = B.charAt(i);  
    if (hm1.containsKey(ch) == false) {  
        hm1.put(ch, 1);  
    }  
    else {  
        int of = hm1.get(ch);  
        hm1.put(ch, of + 1);  
    }  
}
```

HashMap <Character, Integer> hm₂

int match = 0

```
for (i=0; i<K; i++)  
    char ch = A.charAt(i)  
    if (hm2.containsKey(ch) == false) {  
        hm2.put(ch, 1);  
    }  
    else {  
        int of = hm2.get(ch);  
        hm2.put(ch, of + 1);  
    }
```

if (hm₂.get(ch) ≤ hm₁.get(ch))

generating
1st window

match = match + 1;

3

2

if (match == k) { ans = ans + 1 } ;

s = 1 e = k

while (e < n)

// drop A[s-1] add A[e]

char drop = A.charAt(s-1);

if (hm2.get(drop) > hm1.get(drop)) {

// no impact on match, dropped char was invalid

3
else {

match = match - 1; // dropped char was valid

3
hm2.put(drop, hm2.get(drop) - 1);

char add = A.charAt(e);

if (hm2.containsKey(add) == true) {

hm2.put(add, hm2.get(add) + 1);

3

else {

hm2.put(add, 1);

3

if ($hm_2.get(add) \leq hm_1.get(add)$) {

|
3

match = match + 1; // added char is valid

if (match == K) { ans = ans + 1; }

S = S + 1;

e = e + 1;

}

String [] arr = { abc, @bkd, ab }

reference = @bkd

comparison length = min (comparison length vs
arr[i].length())

for (i = 1; i <= 3; i++)

comparison length = 2